

SISTEMA DE INVENTARIO DE REACTIVOS Y CRISTALERÍA

Manual de instalación

Instalación en desarrollo · aplicación Android (APK) · despliegue en producción

Dato	Detalle
Sistema	BioQ — control y gestión del inventario de reactivos y cristalería del laboratorio de química (nombre técnico del código: **LabQuim**)
Estudiante	[Nombre del estudiante] — [Carné]
Carrera	[Carrera]
Documento	Manual de instalación · Examen de Grado
Versión del sistema	0.1.0
Fecha	15 de octubre de 2026

Contenido

Capítulo	Qué resuelve
1. El sistema en una página	Qué es BioQ, qué módulos trae y las formas de usarlo
2. Arquitectura y tecnologías	Cómo se reparten base de datos, API, interfaz web y app Android
3. Requisitos previos	Programas y versiones que hay que tener antes de empezar
4. Obtener el código	Copiar la carpeta del proyecto (ZIP o Git) y limpiarla
5. Base de datos	Crear la base «labquim» con todos los datos o desde los scripts
6. Instalación del backend	API .NET 10: conexión a MySQL, llave JWT y arranque
7. Instalación del frontend	React + Vite: dependencias, dirección de la API y arranque
8. Primer acceso y datos iniciales	Usuarios de arranque y orden para cargar el laboratorio
9. Aplicación Android (APK)	API en la red Wi-Fi, generar e instalar la APK
10. Despliegue en producción	Servidor, configuración de producción, HTTPS y comprobación
11. Problemas frecuentes	Síntoma, causa y solución de los fallos ya vistos
12. Anexos	Configuración, comandos útiles y estructura de carpetas

> **NOTA · Cómo leer este manual**
>
> Los capítulos **3 a 8** instalan el sistema en una computadora de trabajo: es el camino corto (MySQL más dos terminales). El capítulo **9** solo hace falta para la aplicación del teléfono, y el **10** es el servidor real. Cada bloque gris es un comando tal como se escribe en la terminal (PowerShell de Windows); las líneas que empiezan con # son comentarios y no se escriben.

1 El sistema en una página

BioQ controla el inventario del laboratorio de química: los **reactivos** (se consumen, caducan y se controlan por lote, despachando primero el que vence antes — FEFO) y la **cristalería** (material de vidrio que se presta en las prácticas y se devuelve). Las existencias no se editan a mano: todo cambio entra como un movimiento (entrada, salida, ajuste o devolución) registrado en un procedimiento almacenado y en la bitácora de auditoría.

Qué trae instalado

Módulo	Para qué sirve
Seguridad	Inicio de sesión con contraseña cifrada (BCrypt), usuarios con nombres, apellidos y código de estudiante/profesor, cinco roles y permisos editables por rol
Catálogos	Categorías, unidades, proveedores, ubicaciones, laboratorios, clases por año (con sus profesores) y prácticas
Reactivos	Catálogo con ficha química y **rombo de seguridad NFPA 704** obligatorio; existencias por lote y filtros por clasificación de riesgo
Cristalería	Catálogo del material de vidrio y su existencia
Inventario	Existencias actuales, ingreso de stock por lote y alertas de stock mínimo
Solicitudes de reactivos	Profesor o estudiante pide → profesor avala (si la pidió un estudiante) → supervisor aprueba → bodega prepara y entrega (se cierra)
Solicitudes de cristalería	El mismo flujo, pero queda «entregada» hasta que bodega registra la devolución
Reportes	Existencias, próximos a vencer, consumo por laboratorio y kardex, exportables a Excel
Dashboard	Indicadores y gráficos del estado del inventario
Respaldos	Copia y restauración de la base de datos desde la propia interfaz

Dos maneras de usarlo

- **En el navegador** — la forma normal. Basta la base de datos, el backend y el frontend (capítulos 5 a 7).
- **Como aplicación de Android** — una APK instalable, generada con Capacitor a partir de la misma interfaz (capítulo 9).

> OJO · **El servidor siempre es el mismo**
>
> La APK no lleva la base de datos dentro: es la misma interfaz empaquetada y habla con la API por la red. Sin la API corriendo y alcanzable desde el teléfono, la aplicación no muestra datos.

2 Arquitectura y tecnologías

BioQ está separado en tres piezas independientes: la **base de datos** guarda los datos y contiene toda la lógica del negocio en procedimientos almacenados; la **API** solo valida, autentica y llama a esos procedimientos; y la **interfaz** solo consume la API. Por eso pueden vivir en máquinas distintas.

Pieza	Tecnología	Qué hace
Base de datos	MySQL 8 (utf8mb4)	21 tablas y 34 procedimientos almacenados: inventario por lote, flujo de solicitudes, auditoría
Backend (API)	.NET 10 · ASP.NET Core Web API	Rutas REST `/api/*`, permisos por política, documentación Swagger
Acceso a datos	Dapper + MySqlConnector	Llama a los procedimientos y a consultas de lectura
Autenticación	JWT (tokens Bearer) + BCrypt	Al iniciar sesión entrega un token con el rol y los permisos; la interfaz lo envía en cada petición
Excel	ClosedXML	Genera los reportes exportables
Frontend	React 19 + Vite 6 + TypeScript	Toda la interfaz en una sola página (SPA)
Estilos	Tailwind CSS 4 + Flowbite React	Componentes y colores tomados del logo BioQ
Datos y gráficos	TanStack Query · axios · Recharts	Caché de consultas, llamadas a la API y gráficos del dashboard
App Android	Capacitor 8	Empaqueta la interfaz como APK

Cómo se hablan las piezas



Decisiones que conviene conocer antes de instalar

- **La lógica vive en la base de datos.** Crear una solicitud, preparar por FEFO, devolver cristalería o validar el rombo NFPA lo hacen procedimientos almacenados, de forma transaccional. Por eso la base se crea con scripts y no con migraciones de código.
- **Permisos dentro del token.** Al cambiar los permisos de un rol, el usuario debe cerrar sesión y volver a entrar para recibirlos.
- **Zona horaria fija.** La API fija `time_zone = '-06:00'` (Nicaragua) en cada conexión; para eso la cadena de conexión lleva `AllowUserVariables=true`.
- **Respaldos con las herramientas de MySQL.** La API ejecuta `mysqldump.exe` y `mysql.exe`, así que la máquina de la API necesita esos programas (capítulo 10).

3 Requisitos previos

3.1 Para instalar y usar el sistema (obligatorio)

Programa	Versión	Para qué	Cómo obtenerlo
MySQL Server	8.0	Base de datos (incluye mysql.exe y mysqldump.exe)	dev.mysql.com/downloads/installer (incluye MySQL Workbench)
.NET SDK	10	Compilar y ejecutar la API	dotnet.microsoft.com/download/dotnet/10.0
Node.js	20 o superior (probado con 24)	Instalar y ejecutar el frontend	nodejs.org (versión LTS)
Git	Cualquiera reciente	Solo si se trae el código con Git	git-scm.com

Para comprobar que quedaron instalados, en una terminal nueva:

```
dotnet --version
node --version
npm --version
```

3.2 Solo si se va a generar la APK de Android

Programa	Detalle
Java (JDK)	21. Por ejemplo Eclipse Temurin (adoptium.net). La variable `JAVA_HOME` debe apuntar a su carpeta.
Android Studio	Trae el SDK de Android y sus herramientas de compilación.
SDK de Android	Plataforma 36 (Android 16). La APK funciona en teléfonos desde Android 7.0. La variable `ANDROID_HOME` debe apuntar al SDK, normalmente `C:\Users\TU_USUARIO\AppData\Local\Android\Sdk`.

> **NOTA · Gradle se descarga solo**
>
> La primera compilación de la APK descarga Gradle y las dependencias de Android (varios cientos de MB) y tarda varios minutos; las siguientes son mucho más rápidas.

4 Obtener el código

El proyecto es una sola carpeta (`Examen de Grado`) con tres subcarpetas: `database`, `backend` y `frontend`.

Método A — copiar la carpeta o descomprimir el ZIP

1. Copiar la carpeta del proyecto (o descomprimir el ZIP) en una ruta sin caracteres raros, por ejemplo `C:\Proyectos\BioQ\`.
2. Verificar que dentro estén `database\`, `backend\LabQuim.Api\` y `frontend\` directamente (no dentro de una subcarpeta extra).

Método B — con Git

```
git clone URL_DEL_REPOSITORIO BioQ
cd BioQ
```

Limpiar lo que viene de otra computadora

Si la carpeta se copió de otra máquina, conviene borrar estas carpetas generadas; se vuelven a crear solas al compilar:

```
# Desde la raíz del proyecto
Remove-Item -Recurse -Force frontend\node_modules, frontend\dist, `
  backend\LabQuim.Api\bin, backend\LabQuim.Api\obj -ErrorAction SilentlyContinue
```

> **IMPORTANTE · Copiar la carpeta NO copia la base de datos**
>
> Los datos de MySQL viven en el directorio interno de MySQL, no en el proyecto. En la computadora nueva hay que crear la base con el capítulo 5; si no, la API arranca pero no puede trabajar.

5 Base de datos

Todo este capítulo se hace en una terminal abierta en la carpeta `database\`. Hay dos caminos: la base completa con datos (A) o la base vacía desde los scripts (B).

> **IMPORTANTE** · **Siempre con** `--default-character-set=utf8mb4`
>
> Los nombres de reactivos y de personas llevan tildes y ñ. Si los scripts se cargan sin ese parámetro, quedan guardados corruptos (por ejemplo «Ã¡cido» en vez de «ácido») y hay que volver a cargarlos.

5.1 Opción A (recomendada) — base completa en un solo archivo

`labquim_completo.sql` contiene la base entera: estructura, datos, usuarios con sus contraseñas y procedimientos. Crea la base `labquim` desde cero (la borra si ya existía):

```
cd C:\Proyectos\BioQ\database
$mysql = "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe"
& $mysql --default-character-set=utf8mb4 -u root -p -e "source labquim_completo.sql;"
```

Pide la contraseña de `root` de MySQL. Con esto ya está la base; se puede saltar la opción B.

5.2 Opción B — base vacía desde los scripts

Para empezar sin datos de ejemplo (o reconstruir paso a paso). El orden importa:

```
$mysql = "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe"
& $mysql --default-character-set=utf8mb4 -u root -p -e "source 01_schema.sql; source
    02_seeds.sql; source 03_procedures.sql; source 05_procedures_articulos.sql; source
    06_procedures_usuarios.sql; source 07_procedures_catalogos.sql;"
```

Script	Qué crea
01_schema.sql	La base `labquim` y sus 21 tablas
02_seeds.sql	Roles, permisos, catálogos base (unidades, categorías, ubicaciones) y el usuario `admin`
03_procedures.sql	Inventario y flujo de solicitudes (crear, avalar, aprobar, preparar, entregar, devolver)
05_procedures_articulos.sql	Alta y edición de reactivos y cristalería, con validación del rombo NFPA
06_procedures_usuarios.sql	Usuarios (nombres, apellidos, código único) y permisos por rol
07_procedures_catalogos.sql	Catálogos, clases (con sus profesores) y prácticas
04_smoke_test.sql (opcional)	Datos de demostración y una prueba de los dos flujos de solicitud

Para cargar además los datos de demostración:

```
& $mysql --default-character-set=utf8mb4 -u root -p labquim -e "source 04_smoke_test.sql;"
```

> **NOTA · Los scripts 08 a 13 son solo para bases antiguas**
>
> Los archivos `08_migracion_*.sql` a `13_migracion_*.sql` actualizan una base creada con una versión anterior del sistema sin perder sus datos. En una instalación nueva **no** se ejecutan: los scripts 01 a 07 ya incluyen esos cambios.

5.3 Comprobar

```
& $mysql -u root -p labquim -e "SELECT COUNT(*) AS tablas FROM information_schema.tables
    WHERE table_schema='labquim';"
```

Debe responder **21**.

6 Instalación del backend (API .NET)

Todo este capítulo se hace dentro de `backend\LabQuim.Api\`.

6.1 Crear la configuración local

La API lee la conexión a MySQL y la llave de los tokens de `appsettings.Development.json`. Ese archivo contiene contraseñas, por eso **no se sube al repositorio**: si no existe (por ejemplo al clonar con Git), hay que crearlo con este contenido:

BACKEND/LABQUIM.API/APPSETTINGS.DEVELOPMENT.JSON

```
{
  "Logging": {
    "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Information" }
  },
  "ConnectionStrings": {
    "Default": "Server=localhost;Port=3306;Database=labquim;User Id=root;
    Password=TU_CONTRASEÑA;AllowUserVariables=true;"
  },
  "Jwt": {
    "Issuer": "LabQuim.Api",
    "Audience": "LabQuim.Client",
    "Key": "una-clave-larga-y-secreta-de-al-menos-32-caracteres",
    "ExpMinutes": 480
  }
}
```

- Reemplazar `TU_CONTRASEÑA` por la contraseña de `root` de MySQL de esa computadora.
- No quitar `AllowUserVariables=true`: la API lo necesita para fijar la zona horaria.
- La llave JWT debe tener al menos 32 caracteres; si se cambia, las sesiones abiertas se cierran.

6.2 Configuración general

`appsettings.json` ya viene listo. Solo hay que revisar dos cosas:

BACKEND/LABQUIM.API/APPSETTINGS.JSON (FRAGMENTO)

```
"Cors": {
  "Origins": [ "http://localhost:5173", "http://localhost", "https://localhost", "capacitor:
    //localhost" ]
},
"Backup": {
  "Directory": "backups",
  "MySqlBinDir": "C:\\Program Files\\MySQL\\MySQL Server 8.0\\bin"
}
```

- `Cors:Origins`: desde dónde se permite usar la API. El primero es el frontend web; los demás son la app Android.
- `Backup:MySqlBinDir`: carpeta donde están `mysqldump.exe` y `mysql.exe`. Si MySQL se instaló en otra ruta o versión, hay que corregirla o los respaldos fallarán.

6.3 Levantar la API

```
cd C:\Proyectos\BioQ\backend\LabQuim.Api
dotnet run

# Para que un teléfono de la misma red Wi-Fi pueda alcanzarla (capítulo 9):
dotnet run --launch-profile red-local
```

La primera vez restaura los paquetes y compila. Queda escuchando en **http://localhost:5099** y la documentación interactiva de la API está en **http://localhost:5099/swagger**. Hay que dejar esta terminal abierta.

> **NOTA · Contraseñas de desarrollo automáticas**
>
> En modo desarrollo, al arrancar, la API asigna la contraseña `Clave123` a los usuarios cuya contraseña es un marcador inválido (los que crean los scripts 02 y 04). Esto **nunca** ocurre en producción.

7 Instalación del frontend (React + Vite)

Ahora en la carpeta `frontend\`, en una **segunda terminal** (la primera sigue con la API corriendo).

7.1 Instalar las dependencias

```
cd C:\Proyectos\BioQ\frontend
npm install
```

Descarga React, Flowbite, Capacitor y el resto en `node_modules\`, que no viene en el proyecto.

7.2 Indicar dónde está la API

El archivo `frontend\.env.development` ya viene con la dirección de la API local:

```
FRONTEND/.ENV.DEVELOPMENT

VITE_API_URL=http://localhost:5099
```

Solo se cambia si la API corre en otro puerto u otra máquina.

7.3 Arrancar en modo desarrollo

```
npm run dev
```

La aplicación queda en **http://localhost:5173**. La primera vez que se abre una pantalla puede tardar un poco mientras Vite la prepara.

7.4 Generar la versión de producción

```
npm run build

# Para revisar el resultado antes de publicarlo
npm run preview
```

Deja los archivos estáticos en `frontend\dist\`. Eso es lo que se publica en el servidor (capítulo 10) y lo que Capacitor mete dentro de la APK (capítulo 9).

8 Primer acceso y datos iniciales

8.1 Usuarios de arranque

Con la API corriendo en desarrollo, todos estos usuarios entran con la contraseña `Clave123`:

Usuario	Rol	Qué puede hacer
admin	Administrador	Todo: usuarios, roles y permisos, catálogos, inventario, reportes y respaldos
super	Supervisor	Aprobar o rechazar solicitudes; ver inventario y reportes
profe	Profesor	Crear solicitudes y avalar las de sus estudiantes
bode	Bodega	Preparar, entregar y recibir devoluciones; ingresar stock; editar artículos
ana	Estudiante	Crear solicitudes (las avala su profesor) y consultar existencias

Con la opción B del capítulo 5 solo existe `admin` (más `super`, `profe` y `bode` si se cargó el script 04); el resto se crea desde **Usuarios**.

> IMPORTANTE · **Cambiar esas contraseñas antes de usarlo de verdad**
>
> Son contraseñas de demostración, iguales para todos y publicadas en este manual. Al instalar el sistema en el laboratorio hay que asignar contraseñas propias desde **Usuarios → Clave**, o crear cuentas nuevas y desactivar estas. Los usuarios no se borran: se desactivan, para que los movimientos sigan apuntando a quien los registró.

8.2 Orden para cargar el laboratorio

Varias pantallas dependen de otras, así que conviene cargar los datos en este orden:

1. **Catálogos → Categorías, Unidades, Proveedores, Ubicaciones y Laboratorios.**
2. **Usuarios:** profesores y estudiantes con su **código único** (obligatorio para esos dos roles). Nombres y apellidos se guardan en mayúsculas y solo admiten letras.
3. **Catálogos → Clases:** cada clase con su **año** (primero a quinto) y los profesores que la imparten.
4. **Catálogos → Prácticas:** cada práctica pertenece a una clase y se realiza en un laboratorio.
5. **Reactivos:** cada uno con su ficha y su **rombo de seguridad NFPA 704** (salud, inflamabilidad y reactividad de 0 a 4, obligatorios; riesgo específico opcional).
6. **Cristalería:** el material de vidrio.
7. **Ingresar stock** (botón «Ingresar» en cada artículo): crea los lotes, con vencimiento en los reactivos.

> OJO · **Sin clases y prácticas no se pueden crear solicitudes**
>
> Al pedir material se elige profesor → clase → práctica. Si un profesor no tiene clases asignadas, el formulario lo indica y no deja continuar.

8.3 Comprobar que todo quedó bien

- Entrar con `admin` y revisar que el menú lateral muestre todos los módulos.
- Abrir **Reactivos** y **Cristalería**: deben listar los artículos con su existencia.
- Crear una solicitud de prueba y llevarla hasta «Cerrada» con los roles correspondientes.
- Abrir **Respaldos** y crear uno: si falla, revisar `Backup:MySqlBinDir` (capítulo 6.2).
- Exportar un reporte a Excel desde **Reportes**.

9 Aplicación Android (APK)

La APK es la misma interfaz empaquetada con **Capacitor**. El proyecto de Android ya está generado en `frontend\android\`; la aplicación se identifica como `com.bioq.inventario` y se llama **BioQ** en el teléfono.

9.1 Hacer que el teléfono alcance la API

El `localhost` del teléfono es el propio teléfono, así que la APK necesita la **dirección IP de la computadora** en la red Wi-Fi.

1. Averiguar la IP de la computadora con `ipconfig` (línea «Dirección IPv4» del adaptador Wi-Fi), por ejemplo `192.168.1.239`.
2. Escribirla en `frontend\.env.movil`:

```
FRONTEND/.ENV.MOVIL

VITE_API_URL=http://192.168.1.239:5099
```

3. Permitir el puerto 5099 en el firewall de Windows (una sola vez, en PowerShell **como administrador**):

```
New-NetFirewallRule -DisplayName "BioQ API 5099" -Direction Inbound -Protocol TCP -
    LocalPort 5099 -Action Allow -Profile Private
```

4. Levantar la API escuchando en la red: `dotnet run --launch-profile red-local`.

> OJO · **La red Wi-Fi debe estar marcada como «privada»**
>
> La regla anterior aplica a redes privadas. Si Windows tiene la red Wi-Fi como pública, el teléfono no llega a la API aunque todo lo demás esté bien. Se cambia en Configuración → Red e Internet → Wi-Fi → (la red) → Tipo de perfil de red.

9.2 Generar la APK de prueba (debug)

```
cd C:\Proyectos\BioQ\frontend
npm run apk
```

El comando compila la interfaz con la dirección de `.env.movil`, la copia al proyecto de Android y ejecuta Gradle. Al terminar debe mostrar **BUILD SUCCESSFUL** y el archivo queda en:

```
frontend\android\app\build\outputs\apk\debug\app-debug.apk
```

> OJO · **Leer siempre el final de la salida**
>
> Si falta `JAVA_HOME` o `ANDROID_HOME`, Gradle falla antes de compilar y en la carpeta de salida sigue estando la APK de la vez anterior: parece que «no tomó los cambios», cuando en realidad nunca compiló.

9.3 Instalarla en el teléfono

1. Copiar `app-debug.apk` al teléfono (cable, correo o nube) y abrirlo.
2. Android pedirá permitir «instalar apps de fuentes desconocidas»: aceptar.

3. Con el teléfono en la **misma red Wi-Fi** que la computadora y la API corriendo, abrir BioQ e iniciar sesión.

9.4 Ícono y pantalla de inicio

Salen del logo y están en `frontend\assets-app\`. Si se cambia el logo, se regeneran con:

```
npx @capacitor/assets generate --android --assetPath assets-app --iconBackgroundColor
    "#ffffff" --splashBackgroundColor "#ffffff"
```

9.5 APK firmada para distribuir (release)

La APK de prueba basta para usarla y para la defensa. Para publicarla en Play Store o distribuirla oficialmente se firma con un certificado propio:

```
# 1. Crear el certificado (keytool viene con el JDK)
keytool -genkeypair -v -keystore bioq.jks -keyalg RSA -keysize 2048 -validity 10000 -alias
    bioq

# 2. Abrir el proyecto en Android Studio
npx cap open android
# Menú Build > Generate Signed App Bundle / APK > APK > elegir bioq.jks > release
```

> IMPORTANTE · **Si se pierde el certificado, no hay actualizaciones**
>
> Android exige que todas las versiones de una aplicación estén firmadas con el mismo certificado. El archivo `.jks` y sus contraseñas se guardan fuera del proyecto, con copia, y no se suben al repositorio.

9.6 Qué no funciona dentro de la APK

- **Exportar a Excel y descargar respaldos**: usan la descarga del navegador, que el WebView de Android no ejecuta. Esas dos acciones se hacen desde la versión web.
- Al cambiar de red Wi-Fi cambia la IP de la computadora: hay que editar `.env.movil` y volver a generar la APK.

10 Despliegue en producción

Publicar el sistema es poner tres cosas en un servidor: la base de datos MySQL, la API y los archivos estáticos del frontend. El servidor debe ser una máquina donde se puedan ejecutar programas (un VPS o un equipo del laboratorio).

> IMPORTANTE · **Los respaldos necesitan mysqldump y mysql en el servidor**
>
> El módulo de respaldos y restauración ejecuta esas dos herramientas. Muchos hostings compartidos no lo permiten; si ese es el caso, el resto del sistema funciona, pero los respaldos desde la interfaz no.

10.1 Base de datos

Igual que en el capítulo 5 (opción A para llevar los datos actuales, opción B para empezar vacío). En producción conviene crear un usuario de MySQL propio en vez de usar `root`.

> OJO · **La contraseña del administrador en una base nueva**
>
> En producción la API no asigna `Clave123`. Con la opción B el usuario `admin` queda sin una contraseña válida. Solución: arrancar la API una vez en desarrollo contra esa base (asigna `Clave123`), entrar y cambiar la contraseña desde Usuarios; o usar la opción A, que trae las contraseñas actuales.

10.2 Publicar la API

```
cd C:\Proyectos\BioQ\backend\LabQuim.Api
dotnet publish -c Release -o C:\publicar\bioq-api
```

En la carpeta publicada se crea `appsettings.Production.json` con los valores reales (tampoco se sube al repositorio):

APPSETTINGS.PRODUCTION.JSON

```
{
  "ConnectionStrings": {
    "Default": "Server=localhost;Port=3306;Database=labquim;User Id=bioq;
    Password=UNA_CONTRASEÑA_LARGA;AllowUserVariables=true;"
  },
  "Jwt": { "Key": "otra-clave-larga-y-secreta-solo-para-produccion-32+" },
  "Cors": { "Origins": [ "https://bioq.midominio.com" ] },
  "Backup": { "MySqlBinDir": "C:\\Program Files\\MySQL\\MySQL Server 8.0\\bin" }
}
```

Y se ejecuta en modo producción (o se aloja en IIS con el «ASP.NET Core Hosting Bundle» de .NET 10):

```
$env:ASPNETCORE_ENVIRONMENT = "Production"
cd C:\publicar\bioq-api
dotnet LabQuim.Api.dll --urls http://0.0.0.0:5099
```

> OJO · **Cors:Origins es imprescindible en producción**

> Debe contener la dirección exacta desde donde se abre el frontend (y, si se usa la APK, `http://localhost`). Si falta, el navegador bloquea las peticiones y todo parece caído sin un error claro. En producción tampoco se publica Swagger.

10.3 Publicar el frontend

```
cd C:\Proyectos\BioQ\frontend
Set-Content .env.production "VITE_API_URL=https://api.midominio.com"
npm run build
# Se publica el contenido de frontend\dist en el servidor web
```

Es una aplicación de una sola página: el servidor web debe responder `index.html` para cualquier ruta que no sea un archivo (por ejemplo `/reactivos`). En IIS se hace con una regla de reescritura en `web.config`:

FRONTEND/DIST/WEB.CONFIG (IIS)

```
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="SPA" stopProcessing="true">
          <match url=".*" />
          <conditions>
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
          </conditions>
          <action type="Rewrite" url="/index.html" />
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

10.4 HTTPS

Las contraseñas y el token viajan en cada petición: en un servidor real la API y el frontend deben servirse por **https://**, con un certificado válido (por ejemplo Let's Encrypt).

10.5 Lista de comprobación antes de darlo por publicado

Comprobar	Cómo
La API responde	Abrir https://api.midominio.com/api/catalogos/categorias: debe pedir autenticación (401)
El frontend carga y se puede entrar	Abrir el dominio e iniciar sesión con admin
El certificado es válido	El candado del navegador, sin advertencias
Los respaldos funcionan	Crear uno desde Respaldos y descargarlo
La restauración funciona	Probarla en una copia, nunca por primera vez en producción
Las contraseñas de demostración ya no existen	Usuarios con contraseñas nuevas o desactivados
La llave JWT es propia	Distinta a la de desarrollo

11 Problemas frecuentes

Todos estos ocurrieron durante el desarrollo; están con la causa real, no con la aparente.

11.1 Base de datos y backend

Síntoma	Causa	Solución
Tildes corruptas («Ã¡cido»)	Los scripts se cargaron sin utf8mb4	Volver a cargarlos con --default-character-set=utf8mb4
La API no conecta a la base	Contraseña de root incorrecta o el servicio de MySQL detenido	Corregir appsettings.Development.json; iniciar el servicio MySQL80
«Credenciales incorrectas» con Clave123	La API no ha arrancado en modo desarrollo sobre esa base	Arrancar dotnet run una vez (asigna la clave) y volver a intentar
Las fechas salen con una hora de diferencia	Falta AllowUserVariables=true en la conexión	Agregarlo a la cadena de conexión
Crear un respaldo falla	Backup:MySqlBinDir no apunta a mysqldump.exe	Corregir la ruta en appsettings.json
Error de CORS en el navegador	El origen del frontend no está en Cors:Origins	Agregar la dirección exacta y reiniciar la API
Un usuario no ve un módulo tras darle el permiso	Los permisos viajan dentro del token	Cerrar sesión y volver a entrar

11.2 Frontend y uso

Síntoma	Causa	Solución
Pantalla en blanco con un error que no está en el código	El servidor de desarrollo de Vite quedó sirviendo una versión vieja de un archivo	Detener npm run dev y volver a arrancarlo
npm run dev falla al iniciar	node_modules copiado de otra máquina o incompleto	Borrar frontend\node_modules y ejecutar npm install
«Este profesor no tiene clases asignadas»	No hay clases con ese profesor	Registrarlas en Catálogos → Clases y Prácticas
No se puede guardar un reactivo	Falta el rombo de seguridad NFPA	Completar salud, inflamabilidad y reactividad (0 a 4)
No se puede crear un estudiante o profesor	Falta el código o ya lo tiene otro usuario	Asignar un código único (letras, números y guion)

11.3 Aplicación Android

Síntoma	Causa	Solución
En el teléfono no carga ningún dato	La API no está en red, falta la regla del firewall, la red es pública o la IP cambió	`dotnet run --launch-profile red-local`, regla del capítulo 9.1, red privada y revisar `.env.movil`
La APK parece no tomar los cambios	Gradle no llegó a compilar (falta JAVA_HOME o ANDROID_HOME)	Definir las variables y volver a ejecutar npm run apk leyendo el final
«No se instaló la app»	Hay instalada una versión firmada con otro certificado	Desinstalar la versión anterior antes de instalar la nueva
Exportar a Excel no hace nada en el teléfono	El WebView de Android no ejecuta descargas	Exportar desde la versión web
La primera compilación tarda mucho	Gradle descarga sus dependencias	Esperar; las siguientes son rápidas

12 Anexos

12.1 Configuración

Archivo y clave	Ejemplo	Para qué
appsettings.*.json · ConnectionStrings:Default	Server=localhost;Database=labquim;…	Conexión a MySQL
appsettings.*.json · Jwt:Key	(32 caracteres o más)	Firma de los tokens de sesión
appsettings.*.json · Jwt:ExpMinutes	480	Duración de la sesión (8 horas)
appsettings.json · Cors:Origins	http://localhost:5173	Orígenes autorizados a usar la API
appsettings.json · Backup:MySqlBinDir	C:\Program Files\MySQL\…\bin	Carpeta de mysqldump.exe y mysql.exe
frontend/.env.development · VITE_API_URL	http://localhost:5099	API para la versión web en desarrollo
frontend/.env.production · VITE_API_URL	https://api.midominio.com	API para la versión publicada
frontend/.env.movil · VITE_API_URL	http://192.168.1.239:5099	API para la APK

12.2 Comandos útiles

Comando	Qué hace
`dotnet run`	Levanta la API en http://localhost:5099 (desarrollo)
`dotnet run --launch-profile red-local`	La expone a la red local (para el teléfono)
`dotnet publish -c Release -o carpeta`	Compila la API para producción
`npm install`	Instala las dependencias del frontend
`npm run dev`	Frontend en modo desarrollo (http://localhost:5173)
`npm run build`	Compila el frontend a frontend\dist
`npm run apk`	Genera la APK de prueba
`npx cap open android`	Abre el proyecto Android en Android Studio
`mysql … -e "source labquim_completo.sql;"`	Crea la base completa
`mysqldump … --routines --databases labquim`	Exporta la base completa (incluye procedimientos)

12.3 Estructura del proyecto

```
Examen de Grado/
|-- README.md                    Guía rápida de instalación
|-- database/
|   |-- 01_schema.sql ... 07_*   Esquema, datos base y procedimientos (instalación nueva)
|   |-- 08_ ... 13_migracion_*   Actualizan bases de versiones anteriores
|   |-- labquim_completo.sql     Base completa (estructura + datos + procedimientos)
|   \-- ER_diagrama.md           Diagrama entidad-relación (Mermaid)
|-- backend/LabQuim.Api/         API .NET 10
|   |-- Controllers/             Un controlador por módulo
|   |-- Repositories/            Acceso a datos con Dapper (llaman a los procedimientos)
|   |-- Models/                  Peticiones y respuestas, con sus validaciones
|   |-- Auth/                    JWT y permisos por política
|   |-- Services/                Respaldos, Excel y semilla de desarrollo
|   \-- appsettings*.json        Configuración
|-- frontend/                    Interfaz React + Vite
|   |-- src/pages/               Una pantalla por módulo
|   |-- src/components/          Componentes propios (tablas, formularios, rombo NFPA)
|   |-- src/api/                 Llamadas a la API
|   |-- src/assets/logo.jpeg     Logo de la institución
|   |-- assets-app/              Ícono y pantalla de inicio de la APK
|   |-- android/                 Proyecto Android de Capacitor
|   \-- capacitor.config.ts      Nombre e identificador de la app
\-- docs/manual/                 Este manual y su generador
```